Les algorithmes (2/2) : trier, récursivité

Fiche de lecture — tri par sélection, tri à bulles, récursivité, tri par fusion

Kevin Bardoux

Table des matières

1. Le tri 2
- Le problème du tri 2
- Le tri par sélection 2
- Le tri à bulles 3

2. La récursivité 4
- Une fonction qui s'appelle elle-même 4
- La pyramide, deux façons 4
- La factorielle 4

3. Le tri par fusion 5
- Diviser pour régner 5
- L'efficacité du tri par fusion 7

4. Récapitulatif 7
- Dans la vraie vie 7
- À vous de jouer 8

Sources 8

> Note
>
> **Lecture facultative**, environ 15 minutes, en complément de « Recherche, tris & complexité » (semaine 3, jeudi). Début : Les algorithmes (1/2) : chercher, mesurer. D'après CS50, Harvard University, licence CC BY-NC-SA 4.0, leçon 3 (CS50 2026) ; adaptation de Kevin Bardoux, code traduit en JavaScript. Cette fiche est diffusée sous la même licence.

1. Le tri

Le problème du tri

Entrée : un tableau dans le désordre, `5 2 7 4 1 6 3 0`. **Sortie** : le même tableau, trié, `0 1 2 3 4 5 6 7`.

Pourquoi trier ? Pour pouvoir utiliser la recherche dichotomique ; pour afficher des résultats dans un ordre utile (prix, dates, noms) ; et parce que trier une fois, puis chercher des milliers de fois, est très rentable. Trois algorithmes, du plus simple au plus efficace.

Le tri par sélection

On **sélectionne** le plus petit élément et on le met en première position, puis le plus petit du reste en deuxième position, et ainsi de suite.

```
Pour i allant de 0 à n – 2
  Trouver le plus petit élément entre la case i et la fin
  L'échanger avec l'élément de la case i
```

Tour	Plus petit trouvé	Tableau après échange
Départ	—	5 2 7 4 1 6 3 0
1	0	0 2 7 4 1 6 3 5
2	1	0 1 7 4 2 6 3 5
3	2	0 1 2 4 7 6 3 5
…	…	…

```
function selectionSort(array) {
  for (let i = 0; i < array.length - 1; i++) {
    let min = i;
    for (let j = i + 1; j < array.length; j++) {
      if (array[j] < array[min]) min = j;
    }
    const temp = array[i]; // swap, with a temporary variable
    array[i] = array[min];
    array[min] = temp;
```

```
  }
}
```

Efficacité : $(n - 1) + (n - 2) + \ldots + 1 = n(n - 1)/2$ comparaisons, soit $O(n^2)$. Et même si le tableau est déjà trié, il fait toutes ces comparaisons : $\Omega(n^2)$, donc $\Theta(n^2)$.

Le tri à bulles

On compare les **voisins** deux à deux et on les échange s'ils sont dans le mauvais ordre. Les grands nombres « remontent » vers la fin, comme des bulles.

```
Répéter n − 1 fois
  Pour chaque paire de voisins
    S'ils sont dans le mauvais ordre, les échanger
  Si aucun échange n'a eu lieu, s'arrêter
```

Premier passage sur `5 2 7 4 1 6 3 0` : `2 5 7 4 1 6 3 0` (5 et 2 échangés), puis `2 5 4 7 1 6 3 0` (7 et 4 échangés)… jusqu'à `2 5 4 1 6 3 0 7` : le 7 est arrivé à sa place.

```
function bubbleSort(array) {
  for (let i = 0; i < array.length - 1; i++) {
    let swapped = false;
    for (let j = 0; j < array.length - 1 - i; j++) {
      if (array[j] > array[j + 1]) {
        [array[j], array[j + 1]] = [array[j + 1],
array[j]];
        swapped = true;
      }
    }
    if (!swapped) return; // already sorted: stop
  }
}
```

Efficacité : $O(n^2)$ dans le pire des cas. Mais grâce à la variable `swapped`, sur un tableau déjà trié, un seul passage suffit : $\Omega(n)$.

2. La récursivité

Une fonction qui s'appelle elle-même

Une fonction **récursive** résout un problème en s'appelant elle-même sur une version **plus petite** du même problème. La recherche dichotomique en est un exemple : « chercher dans la moitié gauche », c'est… refaire une recherche dichotomique, sur un tableau plus petit. Toute fonction récursive a besoin de deux ingrédients :

- un **cas de base** : une version si simple qu'on connaît la réponse, et qui arrête la récursion ;
- un **appel récursif** sur un problème plus petit, qui finit par atteindre le cas de base.

Sans cas de base, la fonction s'appelle à l'infini, et le programme plante.

La pyramide, deux façons

```
function drawLoop(height) {           // iterative: two
loops
  for (let i = 1; i <= height; i++)
console.log('#'.repeat(i));
}

function drawRec(height) {            // recursive
  if (height <= 0) return;            // base case
  drawRec(height - 1);                // a smaller
pyramid…
  console.log('#'.repeat(height));    // …plus one row
below
}
```

Idée récursive : une pyramide de hauteur 4, c'est une pyramide de hauteur 3, plus une ligne de 4 briques en dessous.

La factorielle

La factorielle de n (notée $n!$) vaut $1 \times 2 \times 3 \times \ldots \times n$. Elle a une définition naturellement récursive : $n! = n \times (n - 1)!$, avec $1! = 1$.

```
function factorial(n) {
  if (n <= 1) return 1;             // base case
  return n * factorial(n - 1);      // recursive call
}
```

Les appels s'empilent jusqu'au cas de base, puis les résultats remontent : 1, puis 2, puis 6, puis 24 (la pile d'appels de la semaine 8).

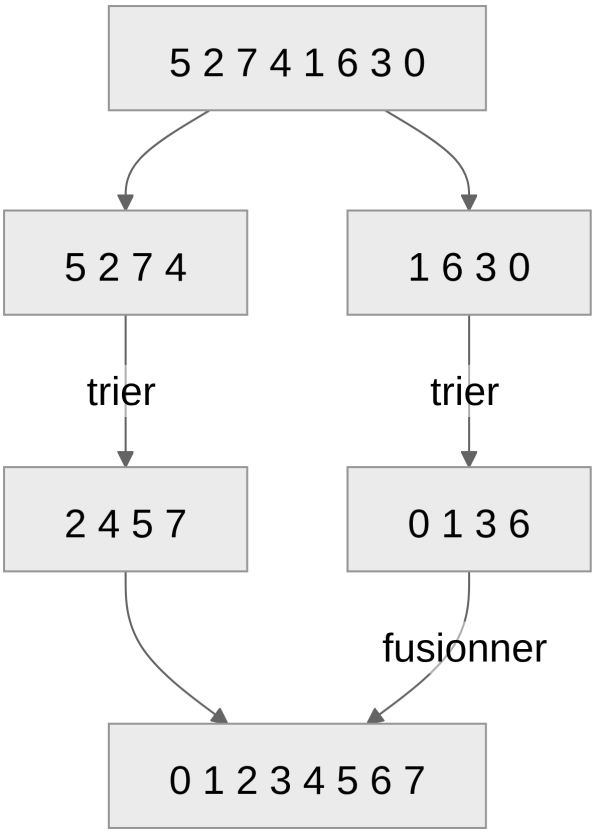
3. Le tri par fusion

Diviser pour régner

Le tri par fusion utilise la récursivité selon le principe **diviser pour régner** :

```
S'il n'y a qu'un seul élément
  Il est déjà trié : s'arrêter
Sinon
  Trier la moitié gauche
  Trier la moitié droite
  Fusionner les deux moitiés triées
```

L'astuce : **fusionner** deux listes déjà triées est très rapide. On compare leurs premiers éléments, on prend le plus petit, et on recommence.



On coupe en deux jusqu'à des éléments seuls, puis on fusionne deux à deux, en gardant l'ordre.

```
function mergeSort(array) {
  if (array.length <= 1) return array;          // 0 or
1 element: sorted
  const middle = Math.floor(array.length / 2);
  const left = mergeSort(array.slice(0, middle));
  const right = mergeSort(array.slice(middle));
  const merged = [];                            //
merge the two sorted halves
  while (left.length > 0 && right.length > 0) {
    merged.push(left[0] <= right[0] ? left.shift() :
right.shift());
  }
  return [...merged, ...left, ...right];
}
```

Le tri par fusion utilise des tableaux temporaires : il consomme **plus de mémoire** que les deux autres.

L'efficacité du tri par fusion

On coupe le tableau en deux jusqu'à des éléments seuls : log n niveaux de découpage. À chaque niveau, la fusion parcourt les n éléments. Total : $O(n \log n)$, et aussi $\Omega(n \log n)$, donc $\Theta(n \log n)$.

Algorithme	Pire cas (O)	Meilleur cas (Ω)	Mémoire en plus
Tri par sélection	n^2	n^2	Aucune
Tri à bulles	n^2	n	Aucune
Tri par fusion	$n \log n$	$n \log n$	Des tableaux temporaires

C'est un compromis classique en informatique : on gagne du temps en utilisant plus d'espace.

4. Récapitulatif

- **Recherche linéaire** : $O(n)$, fonctionne toujours. **Recherche dichotomique** : $O(\log n)$, mais exige un tableau trié.
- La notation **grand O** décrit comment le temps évolue quand les données grandissent ; **Ω** décrit le meilleur des cas, **Θ** quand les deux coïncident.
- Un **objet** regroupe plusieurs données sous un même nom.
- Tri par sélection et tri à bulles : simples, mais en $O(n^2)$. Tri par fusion : $O(n \log n)$, au prix de mémoire supplémentaire.
- Une fonction **récursive** s'appelle elle-même sur un problème plus petit, jusqu'à un cas de base.

Dans la vraie vie

- En JavaScript, `tableau.sort()` trie en place, en $O(n \log n)$; attention, sans paramètre, il compare les nombres comme du texte : `[10, 9, 1].sort()` donne `[1, 10, 9]` (MDN Web Docs, s. d.). Avec une fonction de comparaison : `[10, 9, 1].sort((a, b) => a - b)`.
- Les bases de données utilisent des **index**, triés, pour retrouver une ligne parmi des millions en quelques étapes (semaine 17).

- Les moteurs de recherche, les GPS ou les réseaux sociaux reposent sur des algorithmes efficaces : un algorithme en $O(n^2)$ sur des milliards de données serait inutilisable.

À vous de jouer

Deux exercices, au niveau 3 du parcours de remédiation de la semaine 3 (archive de la semaine) :

1. `f1-dichotomie-recursive` : réécrivez la recherche dichotomique sous forme de fonction récursive.
2. `f2-tri-noms` : triez un tableau de prénoms par ordre alphabétique avec le tri par sélection.

Sources

CS50. 2026. « Lecture 3 - CS50x 2026 ». https://cs50.harvard.edu/x/notes/3/.

MDN Web Docs. s. d. « Array : méthode sort() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/sort.